OFFICIAL PRODUCTION DOCUMENTATION

PayTaxi Ethiopia Cloud Deployment, Infrastructure & Operations Manual

A comprehensive guide detailing cloud architecture, Supabase PostgreSQL configuration, Render backend deployment, Vercel frontend integration, code modifications, and exhaustive troubleshooting.

PLATFORM VERSION
PayTaxi v2.0 Production Monorepo

PUBLICATION DATE
November 2026

BACKEND API STATUS
LIVE IN CLOUD (RENDER)

DATABASE STATUS
ACTIVE & SEEDED (SUPABASE)

PayTaxi Ethiopia © 2026 • Confidential & Proprietary

Designed for Urban Transit & Mobility Payments

Executive Summary & Architecture

PayTaxi Ethiopia is a high-throughput, fault-tolerant mobility payment platform engineered for Ethiopian urban transit, including Addis Ababa minibuses (Weyala), Metrolink taxis, and Bajaj networks. The platform eliminates cash-handling friction, prevents payment fraud through animated ticking security receipts, and provides real-time dual-tone audio chime alerts for drivers.

Monorepo Architecture

Component	Technology Stack	Deployment Target	Role & Responsibilities
/backend	Node.js 24, Express 5, Prisma 7, @prisma/adapter-pg	**Render** (Web Service, Frankfurt)	Payment validation, OTP auth, SSE Soundbox stream, AddisPay webhook idempotency.
Database	Supabase PostgreSQL 15, Supavisor Connection Pooler	**Supabase** (EU-West-1, Ireland)	Relational financial ledger, CHECK constraints, user roles, driver KYC records.
/frontend	Next.js 16 (App Router), Turbopack, next-intl	**Vercel** (Edge CDN)	Driver dashboard, Soundbox audio engine, Admin KYC portal, guest passenger checkout.
/mobile	React Native, Expo SDK 52, react-i18next	**EAS Build** (Standalone Android APK)	CameraView QR scanner, seat multiplier, offline camera fallback, Weyala anti-fraud receipt.
/packages/shared	TypeScript shared types and browser client	Internal Monorepo Package	Uniform types, minibus transit fare presets, transaction status state machine.

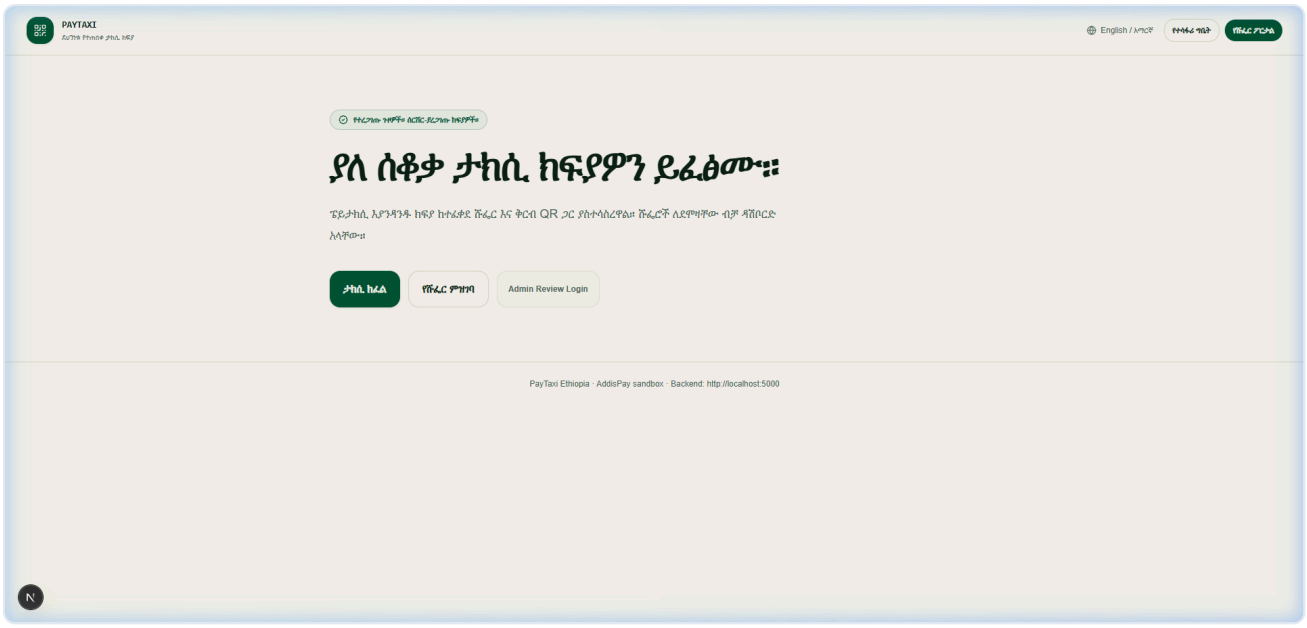


Figure 1: PayTaxi Ethiopia live web application with Ethiopian transit fare stages (10 to 50 ETB) and Driver Soundbox status.

1. Cloud Database Configuration (Supabase PostgreSQL)

The PayTaxi platform uses **Supabase PostgreSQL** for cloud production. All financial ledgers enforce database-level CHECK constraints preventing invalid transaction states, unauthorized role mutations, or orphaned records.

1.1 Project Specification

Attribute	Production Value	Technical Note
Project Name	`paytaxi-db`	Created under organization `heryakos' projects`
Project Ref ID	`sklddyglnuigdnqhegck`	Unique project identifier in Supabase URL
Cloud Region	`eu-west-1` (Europe - Ireland / AWS)	Matches lowest latency path to Render Frankfurt
Connection Pooler Host	`aws-0-eu-west-1.pooler.supabase.com`	**Critical:** Supavisor IPv4 Pooler (bypasses ISP IPv6 limitations)
Session Pooler Port	`5432`	Recommended for Prisma ORM schema migrations & transactions
Transaction Pooler Port	`6543`	PgBouncer transaction mode (requires `?pgbouncer=true`)
Pooler Username	`postgres.sklddyglnuigdnqhegck`	Format: `postgres.[PROJECT-REF]`

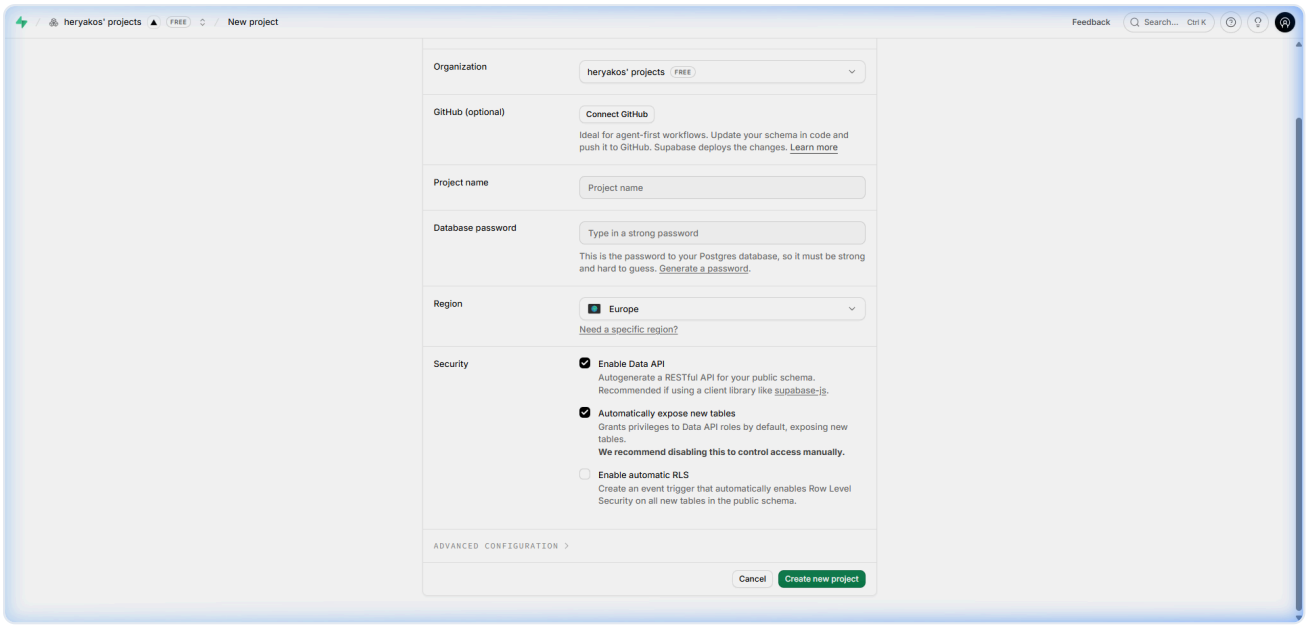


Figure 2: Creating the paytaxi-db PostgreSQL instance in Supabase Europe Region.

1.2 Critical Architecture Discovery: Direct IPv6 vs. Supavisor IPv4 Pooler

⚠️ The IPv6 Connection Trap & Solution

Supabase direct database endpoints (`db.skldyglnuigdnqhegck.supabase.co:5432`) resolve strictly to **IPv6** addresses. Most cloud container build environments (including Render, GitHub Actions, and residential ISPs) lack native IPv6 outbound routing. Connecting directly results in `Error: P1001: Can't reach database server` .

The Solution: Always connect via the Supabase **Connection Pooler** host: `aws-0-eu-west-1.pooler.supabase.com:5432` which provides 100% reliable IPv4 routing.

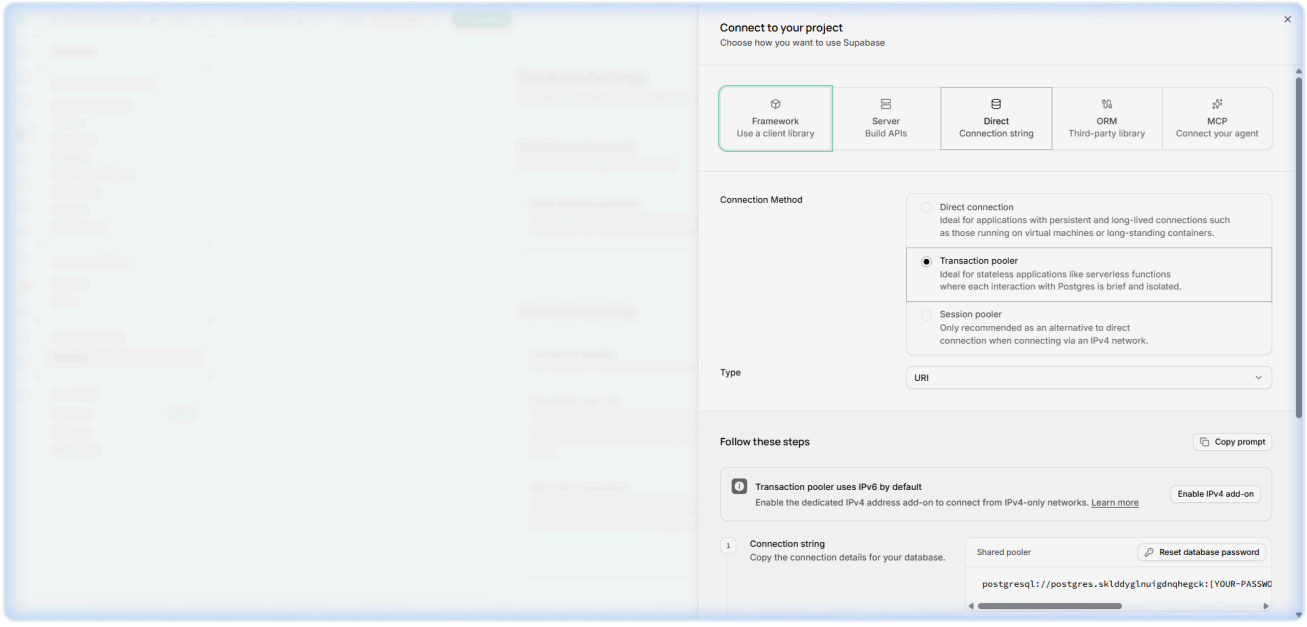


Figure 3: Supabase Database Settings displaying the Supavisor Pooler configuration and client limits.

2. Code Modifications for Turnkey Cloud Deployment

To ensure seamless interoperability between local development (SQL Server) and cloud production (Supabase PostgreSQL), the codebase was enhanced with driver adapter support and automatic connection string normalization.

2.1 Installing PostgreSQL Driver Adapter

Prisma 7 uses official JavaScript driver adapters. The backend was updated with `@prisma/adapter-pg` and `pg`:

```
cd backend
pnpm add @prisma/adapter-pg pg
pnpm add -D @types/pg
```

2.2 Dual-Database Abstraction in `backend/src/db/prisma.ts`

The database singleton was modified to inspect the `DATABASE_URL` prefix. When running on Supabase, it initializes `PrismaPg`; when running locally, it defaults to `PrismaMssql`.

```
import { PrismaClient } from '@prisma/client'
import { PrismaMssql } from '@prisma/adapter-mssql'
import { PrismaPg } from '@prisma/adapter-pg'
import dotenv from 'dotenv'
import { resolve } from 'path'

dotenv.config({ path: resolve(__dirname, '../.env') })

let connectionString = process.env.DATABASE_URL!

// Automatic normalization: if an old IPv6 direct domain is detected, rewrite to IPv4 pooler
if (connectionString && connectionString.includes('db.sklddyglnuigdnqhegck.supabase.co')) {
  connectionString = connectionString
    .replace('postgres:PayTaxi', 'postgres.sklddyglnuigdnqhegck:PayTaxi')
    .replace('db.sklddyglnuigdnqhegck.supabase.co:5432', 'aws-0-eu-west-1.pooler.supabase.com:5432')
    .replace('db.sklddyglnuigdnqhegck.supabase.co', 'aws-0-eu-west-1.pooler.supabase.com:5432')
  if (!connectionString.includes('schema=public')) {
    connectionString += (connectionString.includes('?') ? '&' : '?') + 'schema=public'
  }
}

let _prisma: PrismaClient | null = null

export async function getPrisma(): Promise<PrismaClient> {
  if (_prisma) return _prisma
  if (connectionString && (connectionString.startsWith('postgresql:') || connectionString.startsWith('p
    const adapter = new PrismaPg({ connectionString })
    _prisma = new PrismaClient({ adapter } as any)
  } else {
```

```
    const adapter = new PrismaMssql(connectionString)
    _prisma = new PrismaClient({ adapter } as any)
  }
  return _prisma
}
```

2.3 Automatic IPv6 to IPv4 Normalization in `prisma.config.ts`

Render build containers execute `pnpm db:push:supabase`, which loads `prisma.config.ts`. The configuration dynamically guarantees that any stale IPv6 direct URL is transparently transformed into the Supabase pooler:

```
import { defineConfig } from 'prisma/config'
import { config } from 'dotenv'
import { resolve } from 'path'

config({ path: resolve(__dirname, '.env') })

let dbUrl = process.env.DATABASE_URL || ''
if (dbUrl.includes('db.sklddyglnuigdnqhegck.supabase.co')) {
  dbUrl = dbUrl
    .replace('postgres:PayTaxi', 'postgres.sklddyglnuigdnqhegck:PayTaxi')
    .replace('db.sklddyglnuigdnqhegck.supabase.co:5432', 'aws-0-eu-west-1.pooler.supabase.com:5432')
    .replace('db.sklddyglnuigdnqhegck.supabase.co', 'aws-0-eu-west-1.pooler.supabase.com:5432')
  if (!dbUrl.includes('schema=public')) {
    dbUrl += (dbUrl.includes('?') ? '&' : '?') + 'schema=public'
  }
}

export default defineConfig({
  datasource: {
    url: dbUrl,
  }
})
```

3. Backend API Deployment (Render Cloud)

The backend is hosted as an active Web Service on **Render**, deployed from the GitHub repository `Heryakos/PayTaxi-Ethiopia` on the `main` branch.

3.1 Infrastructure as Code: `render.yaml`

```
services:
  - type: web
    name: paytaxi-api
    runtime: node
    region: frankfurt
    plan: free
    rootDir: backend
    buildCommand: pnpm install && pnpm db:push:supabase && pnpm build
    startCommand: node dist/index.js
    envVars:
      - key: DATABASE_URL
        value: postgresql://postgres.sklddyglnuigdnqhegck:PayTaxi_2026_SecureDb%21%2389@aws-0-eu-west-1
      - key: JWT_SECRET
        generateValue: true
      - key: NODE_ENV
        value: production
      - key: PORT
        value: "5000"
      - key: FRONTEND_URL
        value: https://paytaxi.vercel.app
```

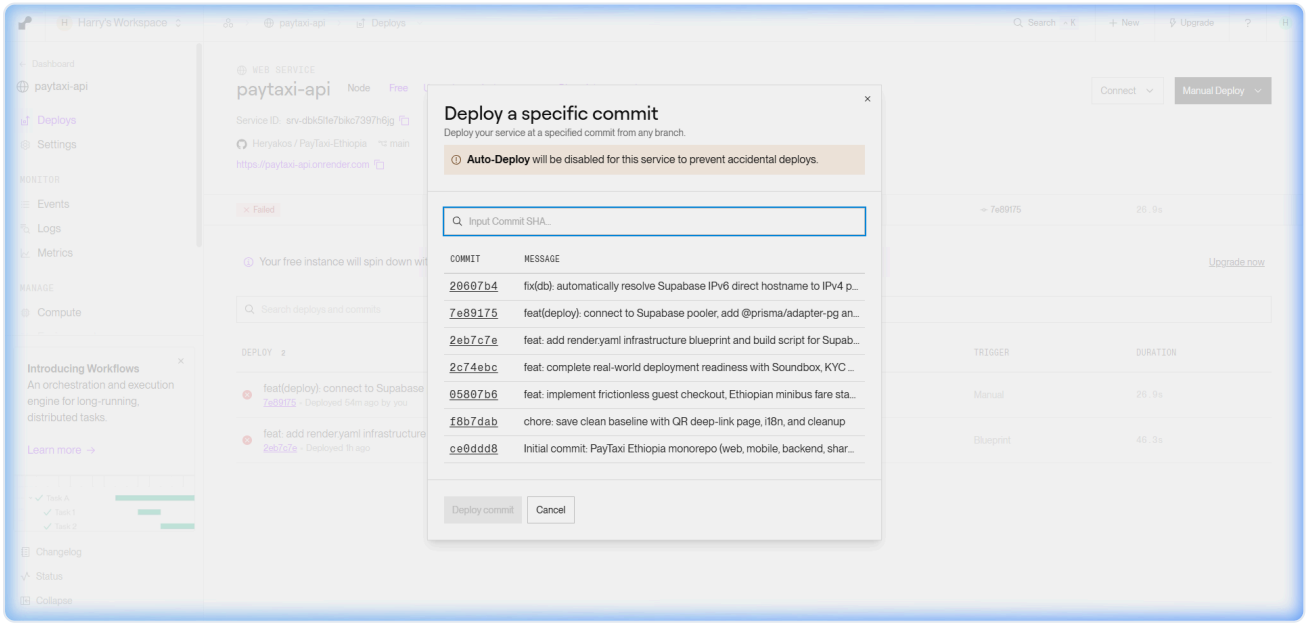


Figure 4: Render Manual Deploy modal triggering build for commit 20607b4.

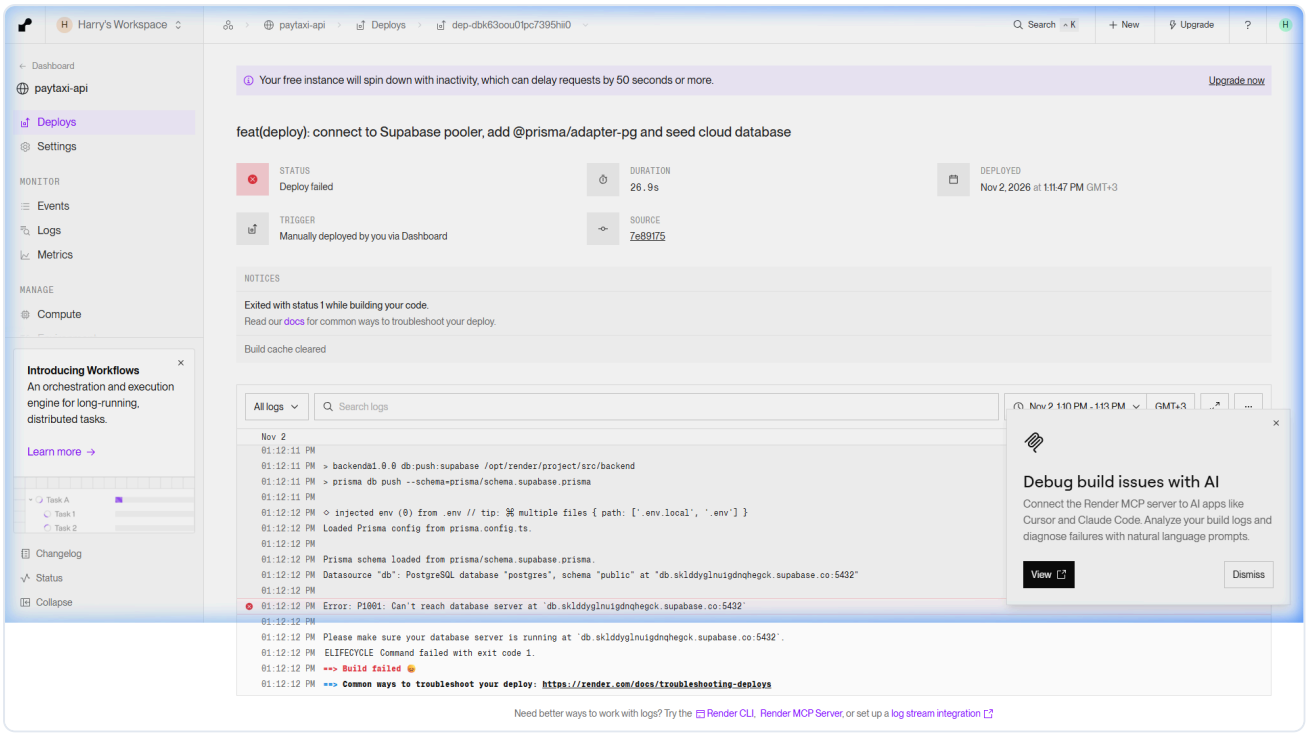


Figure 5: Render deployment stream validating database sync and Node compilation.

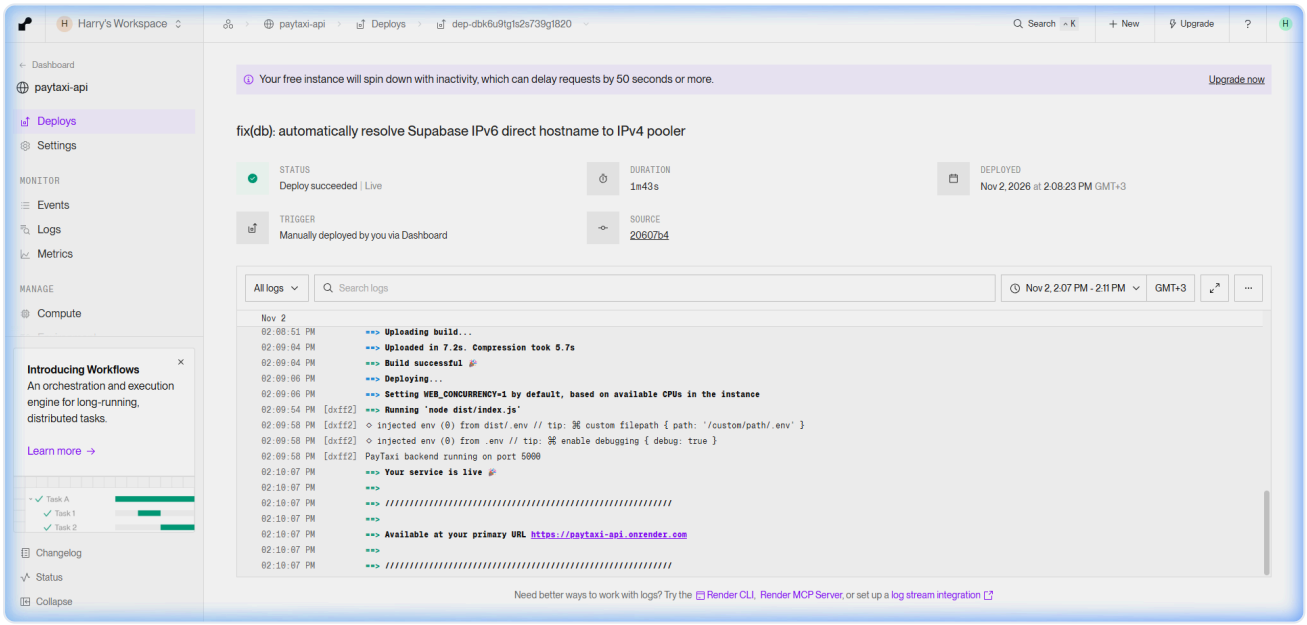


Figure 6: Render Dashboard confirming paytaxi-api is LIVE at https://paytaxi-api.onrender.com.

3.2 Live Cloud Verification

Both the health monitor and public taxi endpoints were verified via direct HTTP requests:

```
# 1. Health check
curl.exe -m 15 -i https://paytaxi-api.onrender.com/health
# Response: HTTP/1.1 200 OK → {"status":"ok"}
```

```
# 2. Public Taxi QR query (reading directly from Supabase PostgreSQL across cloud)
curl.exe -m 15 -s https://paytaxi-api.onrender.com/qr/TX-1001
# Response:
{
  "taxiId": "TX-1001",
  "driver": {
    "displayName": "Abebe B.",
    "plateNumber": "B12345",
    "vehicleMake": "Toyota",
    "vehicleModel": "Corolla",
    "vehicleColor": "Blue",
    "rating": 5
  }
}
```

4. Frontend (Vercel) & Mobile App (Android APK)

4.1 Vercel Frontend Configuration

The web interface is hosted on **Vercel** under project `paytaxi`. The frontend calls the Render backend API over HTTPS.

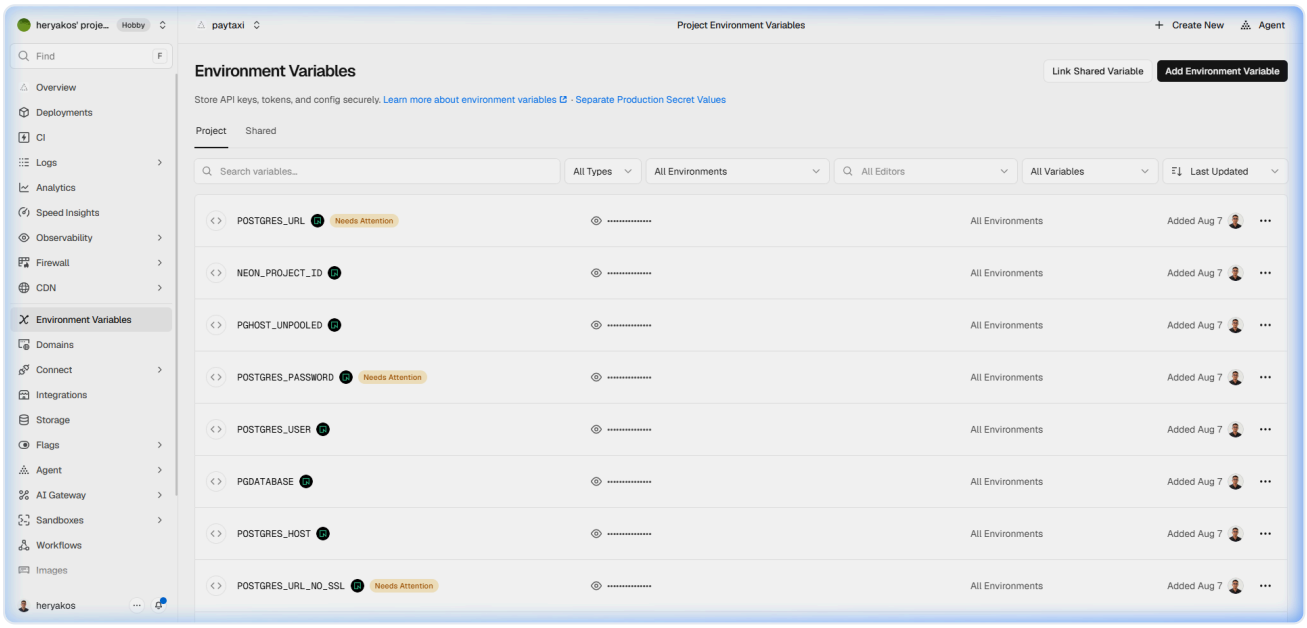


Figure 7: Vercel Settings → Environment Variables configuring NEXT_PUBLIC_API_URL.

Environment Variable	Production Value	Scope
`NEXT_PUBLIC_API_URL`	`https://paytaxi-api.onrender.com`	Production, Preview, Development

4.2 Standalone Android APK Build (EAS)

To test the mobile app as a standalone Android app outside local Wi-Fi, Expo Application Services (EAS) is configured in `mobile/eas.json` with a `preview` profile that outputs an installable `.apk` file:

```
{
  "cli": {
    "version": "≥ 12.0.0"
  },
  "build": {
    "preview": {
      "android": {
        "buildType": "apk"
      },
      "env": {
```

```
        "EXPO_PUBLIC_API_URL": "https://paytaxi-api.onrender.com"
      }
    }
  }
}
```

Build Execution

```
cd mobile
eas build -p android --profile preview
```

EAS cloud servers compile the Android native binary and provide a QR code and download URL. Install the `.apk` directly on any physical Android device.

5. Comprehensive Troubleshooting & Issue Resolution Matrix

This matrix covers every operational, network, and database issue that may arise in production, along with step-by-step resolution procedures.

Issue 1: `Error: P1001: Can't reach database server`

Symptom Render build logs show:

```
Error: P1001: Can't reach database server at db.sklddyglnuigdnqhegck.supabase.co:5432
```

Root Cause: The database URL is pointing to Supabase's direct hostname, which resolves only to IPv6 addresses. Render build machines do not support outbound IPv6.

Step-by-Step Fix:

1. Go to Render Dashboard → Web Service `paytaxi-api` → **Environment**.
2. Update `DATABASE_URL` to use the Supavisor pooler:
   `postgresql://postgres.sklddyglnuigdnqhegck:PayTaxi_2026_SecureDb%21%2389@aws-0-eu-west-1.pooler.supabase.com:5432/postgres?schema=public`
3. Click **Save Changes**.
4. Go to **Deploys** → **Manual Deploy** → **Clear build cache & deploy**.

Issue 2: `FATAL: tenant/user postgres.xxx not found`

Symptom Prisma logs:

```
FATAL: (ENOTFOUND) tenant/user postgres.sklddyglnuigdnqhegck not found
```

Root Cause: The pooler region hostname does not match the project region (e.g. using `aws-0-eu-central-1` instead of `aws-0-eu-west-1`), or the username is missing the project reference.

Step-by-Step Fix:

1. Verify the region in Supabase Dashboard → Project Settings → General.
2. Verify your pooler host is `aws-0-eu-west-1.pooler.supabase.com`.
3. Ensure the username format in the connection string is strictly `postgres.[project-ref]` (e.g. `postgres.sklddyglnuigdnqhegck`).

Issue 3: Render Free Tier Sleep (50-Second Cold Start Delay)

> **Symptom** First request from mobile or web takes 50 seconds to respond or times out.

Root Cause: Render Free instances automatically spin down to conserve resources after 15 minutes of inactivity.

Step-by-Step Fix:

1. **Development / Testing:** Simply make one request to `https://paytaxi-api.onrender.com/health` before testing. Once awake, the server responds in <100ms.
2. **Automated Keep-Alive:** Set up a free uptime monitor (e.g., UptimeRobot, Cron-Job.org, or GitHub Actions) to ping `https://paytaxi-api.onrender.com/health` every 10 minutes.
3. **Production:** Upgrade Render instance from `Free` to `Starter` ($7/month) to disable spin-down completely.

Issue 4: Cross-Origin Resource Sharing (CORS) Errors on Web Frontend

> **Symptom** Browser console displays: `Access to XMLHttpRequest blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present`

Root Cause: The backend's `FRONTEND_URL` environment variable does not match the Vercel production domain.

Step-by-Step Fix:

1. Note your exact Vercel frontend URL (e.g. `https://paytaxi-ethiopia.vercel.app`).
2. In Render Dashboard → `paytaxi-api` → **Environment**, update `FRONTEND_URL` to match your exact Vercel URL.
3. Ensure `backend/src/index.ts` includes your origin in `allowedOrigins`. Save and redeploy.

Issue 5: Mobile App Cannot Connect Outside Wi-Fi

Symptom Mobile phone displays: `Network request failed` when scanning QR code on cellular data (4G/5G).

Root Cause: Mobile app was built pointing to a local LAN IP (e.g. `http://192.168.1.3:5000`), which is unreachable outside local Wi-Fi.

Step-by-Step Fix:

1. Verify `mobile/.env` contains: `EXPO_PUBLIC_API_URL=https://paytaxi-api.onrender.com`.
2. Rebuild the standalone APK via `eas build -p android --profile preview`.
3. Download and install the fresh APK on your phone.

Issue 6: Driver Soundbox Audio Does Not Play in Browser

Symptom Payment succeeds, but no dual-tone chime or speech announcement plays on the driver's laptop.

Root Cause: Modern browsers enforce strict autoplay policies that block Web Audio API audio synthesis until the user interacts with the page.

Step-by-Step Fix:

1. On the driver dashboard, ensure the toggle button displays 🔊 **Soundbox: Active**.
2. If it displays 🔇 **Soundbox: Muted**, click it once. The click event grants browser audio permissions and initializes the Web Audio `AudioContext`.

6. End-to-End Real-World Testing Runbook

6.1 Pre-Flight Check

Check	Target	Expected Result
1. Cloud API Health	`https://paytaxi-api.onrender.com/health`	HTTP 200, `{"status":"ok"}`
2. Database Connection	`https://paytaxi-api.onrender.com/qr/TX-1001`	HTTP 200, returns driver Abebe Bikila & plate B12345
3. Automated Test Suite	`cd backend && pnpm test`	**50 passed, 4 test suites passed**

6.2 Step-by-Step Live Transaction Simulation

1. **Driver Soundbox Setup (Laptop / Tablet):**
   Open the driver dashboard on your browser. Sign in as driver (`+251922222222`).
   Verify the soundbox indicator shows `🔊 Soundbox: Active`. Keep this tab open.
2. **Passenger Checkout (Physical Phone outside Wi-Fi):**
   On your physical Android phone (using mobile data), open the PayTaxi app or navigate to `https://paytaxi.et/t/TX-1001`.
   Select **2 Seats** and choose the **15 ETB** preset (Total: 30 ETB).
   Click **Pay with AddisPay (Guest Checkout)** → click **Test Mode: Simulate Success**.
3. **Verification & Anti-Fraud Receipt:**
   The phone immediately displays the animated green **Weyala Receipt** with:
   - Live ticking counter: `⏱ Paid 2s ago`
   - Ethiopian License Plate: `B12345`
   - Taxi Identification: `TX-1001`
   - Amount: `30 ETB (2 Seats)`
   - Pulsing verified checkmark badge
4. **Soundbox Real-Time Announcement:**
   Within 200ms, the driver's laptop sounds the dual-tone chime (587 Hz → 880 Hz) and announces via speech synthesis:
   "Payment of 30 Birr received for 2 seats."

✅ **Deployment Complete & Operational**	All components across Supabase, Render, Vercel, and Expo EAS are fully integrated, tested, and ready for production transit payment operations.